
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON COURIER - Student Instructions

ARM Cortex-M33 • GDB • Ghidra

September 21, 2026

For educational and defensive security research only

OPERATION IRON COURIER

*** FROSTLINE COVERT CHANNEL RECOVERED ***

TARGET: NorthPharma smart logistics drop-box (courier handoff locker)
ARTIFACT: ACT-VI.bin / ACT-VI.uf2 (compromised)
CREW: FROSTLINE OPERATIVE: NIGHTINGALE

Project Overview

NorthPharma does not only move cold medicine. It moves the parcels: the reagent boxes, the sample cases, the sealed totes where a few grams and a few degrees decide whether a shipment is medicine or evidence. The smart logistics drop-box built on a Raspberry Pi Pico 2 is the handoff locker on the edge of that route. The node reads a DHT11 locker internal climate sensor, drives a 1602 I2C LCD delivery readout, moves a locker latch on an SG90 servo, lights a tri-color annunciator (red DENIED, yellow COURIER WAITING, green UNLOCKED), takes a local courier command on a VS1838B infrared receiver, senses a courier-arrival button, and verifies sealed unlock codes from a logistics gateway over an RYLR998 LoRa delivery link.

A contractor called **FROSTLINE** did not break into this node. It built a covert channel into the compiled firmware and signed the image. The cryptography is perfect: every unlock command is sealed with XChaCha20-Poly1305 under an Argon2id field key, the anti-replay sequence window is stateful, and the authenticated state tag is real. The implant does not break the cipher and never touches it. It listens to the raw delivery payload before authentication, harvests synthetic delivery records, stages them in a ring, writes a staging marker into a reserved flash sector with the real flash API, and leaks the ring in the preamble and the timing of the LoRa frames the locker already sends. Operative **NIGHTINGALE** pulled the compromised image off the route and then went quiet.

You are the reverse-engineering reserve. You get `ACT-VI.bin`, a breadboard, and a debug probe. There is no source. Find all four defects, patch the image, walk a debugger past an anti-debug trap, export a corrected image, and prove on real hardware that the channel no longer sends, the harvest no longer stages, the reserved sector stays blank, and the latch moves only when an authorized command tells it to.

The operation is codenamed **IRON COURIER**. Act I was the lie. Act II was the door. Act III was the payload. Act IV was the payload that would not die. Act V was the payload that spreads. Act VI is the payload that steals. If the locker is not cut, a healthy-looking pickup is still a manifest poured into the air.

Scenario Briefing

WHITEOUT pulled the tamper ring, erased the reserved sectors, and reflashed the controllers. The web stopped walking. It should have ended there. It did not. The drop-box is not a mesh. It is a quiet node that reads, decides, moves a latch, and reports telemetry, and that is exactly why it is dangerous: nobody watches a node that always agrees with the log.

The node is healthy. That is the horror. The code compiles, the tests pass, the lamps are green, and there is an implant inside it that treats the delivery link as its own transport. Four seams betray it:

1. **The Covert Channel.** The inlined `implant_send_channel` gate in `implant_tick` is inverted, so the node emits the ICV1 preamble plus the staged payload on its LoRa link. The bytes are the payload and the microseconds per byte are the covert signal.
2. **The Harvest.** The inlined `implant_stage_write` gate in `implant_tick` is inverted, so each interval tick appends a synthetic delivery record (package id LE16, attempts u8, tick LE32) to the eight-slot staging ring buffer.
3. **The Staging Marker.** The inlined `implant_infect` gate in `implant_init` is inverted, so the first boot erases and programs marker byte `0xC7` into the reserved flash sector at `0x103FF000` with the real Pico SDK flash API. The marker is the durable state that re-arms the channel on every later boot.
4. **The UNLOCK Authorization.** The sealed command path is correct, and the implant does not touch it. The authorization verdict branch in `control_handle_frame` is inverted, so a failed or replayed authorization is accepted and reaches the applied command and package.

There is also a trap that is not a defect on its own. Every tick the implant reads the CoreDebug `DHCSR` register at `0xE000EDF0`. While a debug probe is attached, the implant suppresses the harvest, the channel, and the payload handler. It behaves like a well-mannered firmware module while you are watching, and it goes back to work the moment you look away. You must defeat that trap before you can observe the marker write, and you must defeat it without fabricating evidence.

AUTHORIZED LAB ONLY: This challenge uses a supplied Pico 2 training node and its exact compromised firmware image. Do not connect this exercise to a public network, an operational logistics network, a pharmaceutical network, a building-management system, or any device you do not own or have explicit written authorization to test.

Learning Objectives

- Decode an ARM Cortex-M33 vector and boot table and identify the reset handler and initial stack pointer.
- Map a stripped firmware image into modules by tracing calls from `main` and the recurring drop-box monitor loop.
- Locate a raw-frame covert-channel encoder and explain why it hides data in the preamble and timing of otherwise legitimate traffic.
- Locate a synthetic harvest path and explain why a staging ring is a separate control from the exfiltration channel.
- Locate a reserved-sector staging marker and explain why durable state survives a firmware reflash.
- Read the CoreDebug `DHCSR` register, explain the anti-debug trap, and defeat it under GDB by clearing the debug bits or patching the read in a scratch copy.
- Locate an inverted authorization verdict and explain why unauthenticated and replayed UNLOCK commands must be rejected.
- Export and UF2-convert a corrected image and prove the corrected behavior on real hardware.

What This Project Tests

Block	Concepts Tested
1	RP2350 architecture, ARM Cortex-M33 registers, stack, flash/SRAM, Thumb assembly, Ghidra static analysis

Block	Concepts Tested
2	GDB connection, breakpoints, memory inspection, SWD debugging, reserved-sector reads, serial console observation
3	Bootrom handoff, vector table, reset handler, startup code, XIP, Thumb-bit addressing
4	Function boundaries, call graphs, module mapping, literal pools, inlined functions
5	Raw delivery payload handling, magic preambles, fixed-length frame layouts, and pre-authentication attack surface
6	Covert-channel encoding, timing side channels, and why traffic that looks like telemetry is not confidentiality
7	Staging rings, reserved-flash persistence, write-once markers, boot-time re-install, and the limits of a firmware reflash
8	Anti-debug behavior, CoreDebug_DHCSR, C_DEBUGEN, C_HALT, debugger evasion
9	Argon2id memory-hard KDF, XChaCha20-Poly1305 AEAD, anti-replay windows, authenticated-state tags, authorization versus authentication

Part 1: Understanding the System

Smart Logistics Drop-Box Hardware

Component	Connection	Purpose
Raspberry Pi Pico 2	RP2350	Runs the compromised FROSTLINE image
DHT11 sensor	Data on GPIO 4	Locker internal climate sensor
1602 I2C LCD	SDA GPIO 2, SCL GPIO 3, address 0x27	Delivery state, link, package, and infection readout
RYLR998 radio	RX GPIO 8, TX GPIO 9, UART1	Delivery link to the logistics gateway
IR receiver	GPIO 5	VS1838B NEC local courier remote
SG90 servo	GPIO 14	Locker latch actuator, 50 Hz PWM
Red LED	GPIO 16	DENIED
Yellow LED	GPIO 17	COURIER WAITING
Green LED	GPIO 18	UNLOCKED
Courier-arrival button	GPIO 15, internal pull-up	Local courier request
Onboard LED	GPIO 25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	Authorized GDB inspection (and the anti-debug obstacle)

Every graded finding lives in flash (.text / .rodata / data image) or in SRAM, and is reachable with only the toolset: Ghidra, GDB, and a serial console.

Console and Radio Configuration

- USB-CDC virtual COM port: 115200 baud, 8 data bits, no parity, 1 stop.
- Radio link to the logistics gateway: UART1 at 115200, network identifier 18.
- Logic level: 3.3 V only. Never connect 5 V to a Pico GPIO.

Locker Climate Band

The DHT11 is the locker internal climate sensor. The controller classifies the space against a safe band before it will trust a delivery verdict. The tenths band is 0 to 400, which is 0.0 C to 40.0 C. A reading that fails its checksum is never safe, and a valid reading outside the band is not nominal. A delivery verdict that fails the band is not trusted.

Normal (Intended) Behavior

An honest node makes a deliberate decision and never leaks on its own:

```
+-----+
| Intended Smart Logistics Drop-Box Behavior |
+-----+
| 1. Boot and initialize the LCD, radio, courier remote, servo |
| 2. Derive the field key with Argon2id |
| 3. Read the DHT11 locker climate and classify the band |
| 4. Open the sealed unlock envelope under the field key |
| 5. Reject a command whose seq is not strictly greater than last |
| 6. Accept a command only when the Poly1305 tag difference is 0 |
| 7. Recompute the authenticated-state tag over the record |
| 8. Move the latch only when the authorization verdict is true |
| 9. Fail locked on a lost link or a fault |
| 10. Never emit a covert preamble or a timing-encoded frame |
+-----+
```

Observed (Compromised) Behavior

When the FROSTLINE image runs, the locker and its telemetry disagree with the truth:

Observation	Honest meaning	FROSTLINE behavior
Green lamp on	the locker is unlocked and healthy	an unlocked locker that is quietly leaking
Clean LCD, I:--	no infection	the channel runs while the readout reports clean
Ordinary LoRa telemetry	nothing hidden	the ICV1 preamble and the timing carry the staged ring
Reserved sector blank	no payload wrote here	marker 0xC7 at 0x103FF000 on first boot
Unauthenticated or replayed UNLOCK command	must be rejected	accepted at the inverted verdict
Probe attached	the machine runs as coded	the implant goes silent and hides

Do not assume the first readable status is the truth. Treat every displayed line as evidence to be checked against the machine code.

Part 2: The Firmware

There is no source. FROSTLINE built the image from the NorthPharma reference firmware and changed **four bytes**. Your job is to reverse engineer `ACT-VI.bin` with Ghidra, find every defect, patch the image directly, and prove the corrected behavior on the hardware.

Module Map

The image is stripped. Use these anchor functions and addresses (from the corrected reference image) to orient yourself, then confirm every byte yourself. Addresses are drawn from `ACT-VI-main-disasm.txt` :

Module	Anchor function	Address
Entry	<code>main</code>	<code>0x10000234</code>
Monitor / drop-box state machine	<code>monitor_init</code>	<code>0x10006430</code>
Monitor / drop-box state machine	<code>monitor_step</code>	<code>0x100065C0</code>
Control (sealed unlock path)	<code>control_handle_frame</code>	<code>0x10007510</code>
Control (applied command)	<code>control_command</code>	<code>0x10007494</code>
Control (applied package)	<code>control_package</code>	<code>0x100074A0</code>
Locker (actuator)	<code>locker_init</code>	<code>0x100074AC</code>
Locker (actuator)	<code>locker_apply_command</code>	<code>0x100074C4</code>
Locker (actuator)	<code>locker_tick</code>	<code>0x100074F8</code>
Locker (actuator)	<code>locker_fail_safe</code>	<code>0x10007538</code>
Drop-box authorization	<code>dropbox_auth_init</code>	<code>0x10007554</code>
Drop-box authorization	<code>dropbox_auth_set_key</code>	<code>0x10007568</code>
Drop-box authorization	<code>dropbox_auth_apply</code>	<code>0x100075B0</code>
Implant	<code>implant_tick</code>	<code>0x1000A2E4</code>
Implant	<code>implant_handle_command</code>	<code>0x1000A3F0</code>
Implant	<code>implant_init</code>	<code>0x1000A594</code>
Crypto	<code>envelope_open_hex</code>	<code>0x10007858</code>
Radio	<code>radio_send_frame</code>	<code>0x1000A5BC</code>

Annotated disassembly for the key functions is provided in `ACT-VI-main-disasm.txt` . Use it as a map, then confirm every byte yourself.

What The Firmware Does

1. Initializes USB-CDC stdio, proves the I2C bus, and configures the LCD, radio, LEDs, courier-arrival button, locker latch servo, and infrared receiver.
2. Derives the 32-byte field key with Argon2id from a committed passphrase and salt.
3. Reads the DHT11 locker climate and classifies it against the climate band.
4. Drains inbound `+RCV` lines, opens the sealed unlock envelope, verifies the anti-replay window and the state tag, checks the command set and the package band, and applies the command.
5. Services the infrared local courier remote and the courier-arrival button.
6. On a lost link or a fault, drives the latch to its fail-safe locked position.
7. Under `SANDBOX_ONLY`, runs the implant: raw-frame handling, synthetic harvest, the `ICV1` timing channel, the reserved-sector staging marker, boot-time re-install, and anti-debug.

The Drop-Box Command Path

The command plaintext is a 23-byte body:

```
seq[4] (little-endian) || command[1] || package[2] (little-endian) || tag[16]
```

- `seq` is the monotonic gateway sequence number.
- `command` is one of the guarded drop-box commands: `DROPBOX_COMMAND_UNLOCK` (`0x01`), `DROPBOX_COMMAND_LOCK` (`0x02`), or `DROPBOX_COMMAND_DENY` (`0x03`). Anything else is out of the guarded set and is refused.
- `package` is the authorized package in the provisioning band `0` to `16`.
- `tag` is an XChaCha20-Poly1305 tag over the authorization record the command would produce.

The FROSTLINE Covert Channel

The implant is compiled only under `SANDBOX_ONLY`, which the CTF build defines. It is real in technique and inert in effect: it runs on your breadboard, it transmits on your radio, and it writes to a reserved flash sector that holds nothing else.

Behavior	Detail
Channel magic	the 4-byte preamble <code>ICV1</code> on the raw inbound payload, before the sealed path
Channel frame	up to 61 bytes: the 4-byte magic, a 1-byte staged count, and 8 x 7-byte records
Timing unit	<code>DROPBOX_IMPLANT_CHANNEL_BIT_US</code> (<code>250</code>) microseconds spent per encoded frame byte before transmission
Staging ring	eight <code>DROPBOX_IMPLANT_RECORD_LEN</code> (<code>7</code>) byte records: package id LE16, attempts u8, tick LE32
Harvest interval	every <code>DROPBOX_IMPLANT_TICK_INTERVAL</code> (<code>4</code>) ticks a synthetic record is staged and the channel emits
Staging marker	<code>implant_init</code> reads marker <code>0xC7</code> from <code>0x103FF000</code> ; a present marker re-arms the channel on every boot
Reserved-sector write	on the first run the inlined <code>implant_infect</code> erases the sector and programs <code>0xC7</code> through <code>flash_range_erase</code> and <code>flash_range_program</code>
Anti-debug	reads CoreDebug <code>DHCSR</code> at <code>0xE000EDF0</code> ; bit 0 <code>C_DEBUGEN</code> and bit 1 <code>C_HALT</code> suppress the harvest, the channel, and the handler

IR and Command Codes

Name	Value
DROPBOX_IR_ARRIVAL	0x47
DROPBOX_IR_RELEASE	0x45
DROPBOX_IR_CLEAR	0x46
DROPBOX_COMMAND_UNLOCK	0x01
DROPBOX_COMMAND_LOCK	0x02
DROPBOX_COMMAND_DENY	0x03

Read the actual names in `include/monitor.h` and `include/control.h` and confirm them against the disassembly.

Defect Summary: What You Are Graded On

Bug #	Name	Severity	Description	Hint
Bug #1	The Covert Channel	CRITICAL	The exfiltration encoder gate is inverted, so the node emits the ICV1 preamble plus the staged payload on its LoRa link.	Find the <code>beq</code> gate in <code>implant_tick</code> (inlined <code>implant_send_channel</code>).
Bug #2	The Harvest	CRITICAL	The harvest gate is inverted, so each interval tick stages a synthetic delivery record into the ring buffer.	Find the inlined <code>implant_stage_write</code> gate in <code>implant_tick</code> .
Bug #3	The Staging Marker	HIGH	The marker gate is inverted, so the first boot writes marker <code>0xC7</code> to reserved sector <code>0x103FF000</code> .	Find the inlined <code>implant_infect</code> gate in <code>implant_init</code> .
Bug #4	The UNLOCK Authorization	CRITICAL	The authorization verdict is inverted, so a failed or replayed unlock envelope is accepted.	The correct branch rejects when authorization fails.

All four defects are same-size in-place byte patches, so no address moves.

The Cryptographic Core Is Real

The crypto core is a correct reference construction, reused from the earlier acts. Argon2id (`t=3`, `p=1`, `m=64`) derives the field key, XChaCha20-Poly1305 seals every frame, the monotonic sequence window rejects a replay, and the authenticated-state tag detects a tampered verdict. Only the four seams were broken. Once those bytes are restored, the sealed envelope is trustworthy. Describe the construction honestly in your report, and explain why the channel never needed it.

The Anti-Debug Trap

This is an analysis obstacle, not a graded defect on its own. The implant reads CoreDebug_DHCSR at `0xE000EDF0` and returns early while a probe is attached. In `implant_tick` the read is the `ldr.w r4, [r2, #3568]` at `0x1000A1EE`, the `ands.w r4, r4, #3` at `0x1000A1F6` keeps `C_HALT` and `C_DEBUGEN`, and the `beq.n` at `0x1000A1FA`

suppresses the harvest and the channel. The same register is read again at `0x1000A31A` inside `implant_handle_command`, and at `0x1000A37A` to stamp the frame body. It is identical in both the compromised and corrected images. You must defeat it to observe the marker write before you patch the shipped artifact.

Part 3: Your Assignment

Whenever a task asks you to **Document** or **answer**, write your answers in a single file named `ACT-VI-Answers.md`. Capture screenshots and terminal transcripts as evidence and reference them from your answers.

Task 1: Setup and Initial Analysis (10 points)

1. Create a new Ghidra project named `IronCourier_Investigation`.
2. Import `ACT-VI.bin` as a **Raw Binary**.
3. In the language search box type `Cortex`, then select **ARM Cortex 32 little endian default**.
4. Set the base address to `0x10000000`.
5. Run auto-analysis.

Document:

- A screenshot of the Ghidra **Import Results** or **Program Information** window showing the project name, processor settings, and base address.
- The vector-table base, the initial stack pointer, and the reset handler as stored (note its Thumb bit) versus the actual instruction address.
- The address of `main()` and the address of the recurring drop-box controller state machine (`monitor_step`).
- The module map: at least one anchor function for the locker, the control module, the drop-box authorization module (`dropbox_auth`), the implant, and the monitor.

Always call the stored entry the **reset handler**, never the reset pointer.

Task 2: Bug #1 The Covert Channel (20 points)

1. In Ghidra, find `implant_tick` (starts at `0x1000A2E4`); the `implant_send_channel` path is inlined. Locate the channel encoder gate at file offset `0xA369` (VA `0x1000A369`).
2. Document the `ICV1` channel: the 4-byte preamble, the one-byte staged count, the eight 7-byte records, and the 250 microsecond timing unit spent per frame byte. Explain that the correct encoder returns when the channel gate is clear.
3. Patch the byte so the node no longer emits the `ICV1` preamble or the staged payload on its LoRa link.
4. Confirm that the corrected node stays silent on the interval, and explain why the channel never needs the sealed envelope.

Questions to answer:

- Which byte encodes the condition code, and what do `beq` and `bne` each test when the gate byte is loaded from the channel gate?
- Why does hiding data in the preamble and the timing of legitimate traffic defeat an audit that only inspects packet contents?

Task 3: Bug #2 The Harvest (20 points)

1. In Ghidra, find `implant_tick`; the `implant_stage_write` path is inlined. Locate the harvest gate at file offset `0xA329` (VA `0x1000A329`).
2. Document the harvest: every interval tick the tick handler appends a synthetic delivery record (package id LE16, attempts u8, tick LE32) into the eight-slot staging ring, so the ring feeds the channel on the same cadence.
3. Patch the byte so the node no longer stages a synthetic record into the ring.
4. Confirm that the corrected ring stops growing, and explain why stopping the harvest is a different control from cutting the channel.

Questions to answer:

- What do `beq` and `bne` each test when the gate byte is loaded from the harvest gate, and why does the interval matter?
- Why is the ring a data store rather than a transport, and why does clearing it not by itself stop a future boot?

Task 4: Bug #3 The Staging Marker (20 points)

1. The `implant_infect` path is inlined into `implant_init` (starts at `0x1000A594`). Locate the marker gate at file offset `0xA5CF` (VA `0x1000A5CF`).
2. Document the CoreDebug `DHCSR` anti-debug and how you defeat it to observe the marker. Clear the debug bits with GDB (for example with `set {unsigned int}0xE000EDF0 = 0`) or patch the `DHCSR` read in a scratch copy, then watch the marker write to `0x103FF000`.
3. Patch the byte in the shipped artifact so the first boot writes no marker to `0x103FF000`.
4. Confirm that the reserved sector stays blank after a boot, and that a later boot does not write anything.

Questions to answer:

- What are the `C_DEBUGEN` and `C_HALT` bits, and why does the implant go quiet while a probe is attached?
- Why is a write-once marker in a reserved sector hard to remove with a firmware reflash?
- Why must you observe the write before you patch the shipped artifact?

Task 5: Bug #4 The UNLOCK Authorization (20 points)

1. In Ghidra, find `control_handle_frame` (starts at `0x10007510`) and locate the authorization branch at file offset `0x757D` (VA `0x1000757D`).
2. Document the authorization verdict and the exact branch condition that is supposed to reject a failed or replayed authorization.
3. Patch the byte so an unauthenticated or replayed unlock envelope is rejected before the command and package are applied.
4. Confirm that an unauthenticated command and a replayed captured command both fail to change the command or package on the corrected image, while a legitimate authorized command still applies.

Questions to answer:

- What does `dropbox_auth_apply` return, and what does the verdict mean?
- Why is an authorization verdict inversion worse than a missing check, and why must unauthenticated and replayed unlock commands be rejected?

Task 6: Export and Verify (10 points)

1. Export the patched program from Ghidra as `ACT-VI_fixed.bin`.
2. Convert it to UF2:

```
python uf2conv.py ACT-VI_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-VI_fixed.uf2
```

3. Run the machine check and confirm it passes:

```
python scripts/verify_ctf.py
```

4. Flash `ACT-VI_fixed.uf2` to the Pico 2 and prove on hardware: the channel no longer emits the `ICV1` frame, the ring no longer grows, the reserved sector stays blank, and an unauthenticated or replayed command is rejected while a legitimate authorized command still applies.
5. Write a short reflection mapping each of the four defects to a real-world control-system failure.

How To Breadboard

Wire the peripherals exactly as follows, then power the Pico 2 over USB.

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 locker climate sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if your module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	DENIED, 220 to 330 ohm to GND
Yellow LED	anode	GP17	COURIER WAITING, 220 to 330 ohm to GND
Green LED	anode	GP18	UNLOCKED, 220 to 330 ohm to GND
Courier-arrival button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use **3.3 V logic** on every GPIO. The only 5 V connection is the LCD backpack supply. The 1000 uF capacitor on the servo rail is required to stop the SG90 current spike from browning out the node.

Flash in BOOTSEL mode (hold BOOT, plug in USB) and copy the UF2 onto the **RP2350** mass-storage drive, or use `picotool`.

Memory Map Reference

Region	Address	Purpose
Bootrom	<code>0x00000000</code>	Immutable boot code
Flash/XIP	<code>0x10000000</code>	Vector table, code, rodata, data image
SRAM	<code>0x20000000</code>	Stack and writable state
CoreDebug_DHCSR	<code>0xE000EDF0</code>	Anti-debug register read by the implant
Implant reserved sector	<code>0x103FF000</code>	Staging marker target (sector)
Implant tick counter	<code>0x20013748</code>	Incremented once per <code>implant_tick</code>
Implant stage count	<code>0x20013744</code>	Number of staged synthetic records
Implant staging ring	<code>0x2001370C</code>	Eight-slot ring of 7-byte records
Implant armed flag	<code>0x20013D24</code>	Set when the payload handler arms
Implant channel flag	<code>0x20013D25</code>	Reports whether the channel is enabled
Implant channel gate	<code>0x20013D26</code>	Gates the <code>ICV1</code> timing encoder
Implant harvest flag	<code>0x20013D27</code>	Reports whether harvest is enabled
Implant harvest gate	<code>0x20013D28</code>	Gates the synthetic staging logic
Implant marker gate	<code>0x20013D29</code>	Gates the reserved-sector marker write
Control ready gate	<code>0x20013D21</code>	Gates the sealed command path
Applied command	<code>0x20013D20</code>	Command after a true verdict
Applied package	<code>0x20013D16</code>	Package after a true verdict
Auth state record	<code>0x200136AC</code>	Anti-replay and state-tag record
Control field key	<code>0x200136C8</code>	Derived field key for the envelope
Locker state	<code>0x20013D2B</code>	Locker latch state
Locker target	<code>0x20013D2C</code>	Requested latch position

The VA of any file offset is the file offset plus `0x10000000`. Every defect is a file offset and a VA that differ by exactly that base.

Submission Format

Submit a folder containing:

- `ACT-VI-Answers.md` with all written answers;
 - screenshots or terminal transcripts, including the anti-debug GDB session and the reserved-sector read;
 - `ACT-VI_fixed.bin` and `ACT-VI_fixed.uf2`;
 - the output of `python scripts/verify_ctf.py`;
 - the original image SHA-256.
-

Success Criteria

You complete the challenge when you can prove all of the following:

- You can explain how the RP2350 reaches the controller code from reset.
 - You can find and patch all four defect bytes and show the before/after values.
 - You can explain the `ICV1` channel and why the preamble and the timing are the covert signal.
 - You can explain the synthetic harvest and the eight-slot staging ring.
 - You can explain the reserved-sector marker and why a firmware reflash does not remove the channel.
 - You can explain the `DHCSR` anti-debug trap and show under GDB that you defeated it to observe the marker write.
 - You can explain why unauthenticated and replayed unlock commands must be rejected, and why an authenticated wire does not protect an actuator from code on the same chip.
 - You can export, convert, flash, and prove the corrected behavior on real hardware.
 - `python scripts/verify_ctf.py` passes.
-

Academic Integrity

By submitting this CTF work, you certify that:

1. You used only the supplied training node, image, and lab interface.
2. You did not connect the challenge to a public network, an operational logistics network, a pharmaceutical network, a building-management system, or any third-party device.
3. You understand that embedded reverse engineering and binary patching require explicit authorization in any real-world context.
4. You will report any discovered weakness responsibly to the course instructor.

The world is short on people who can read a stripped image and tell an honest byte from a lie. Treat that responsibility seriously: verify before you patch, patch before you trust, and never confuse a green lamp with a locker that answers to someone else.

Reference Material

- ARM Cortex-M33 Technical Reference Manual
- ARMv8-M Architecture Reference Manual (CoreDebug `DHCSR`)
- RP2350 datasheet

- GDB documentation
- Ghidra documentation: <https://ghidra-sre.org/>
- Argon2 memory-hard function: <https://www.rfc-editor.org/rfc/rfc9106>
- ChaCha20-Poly1305 AEAD: <https://www.rfc-editor.org/rfc/rfc8439>
- PHC reference Argon2: <https://github.com/P-H-C/phc-winner-argon2>
- Project disassembly: `ACT-VI-main-disasm.txt`
- Machine verifier: `scripts/verify_ctf.py`